Assignment - 8

Implement a New Congestion Control Algorithm in NS3

SLOK TULSYAN

12342080

23 March 2026

Question 1

A new TCP variant named **TcpSlok** was implemented by extending the `TcpCongestionOps` class in ns-3.

- **Key Design Features:**
  - Inherits behavior from standard TCP
  - Overrides:
    - `IncreaseWindow()` → for congestion window update
    - `GetSsThresh()` → for loss handling
    - `PktsAcked()` → for RTT tracking
- **Objective:** To improve TCP performance by making the Slow Start phase **adaptive and intelligent**, instead of blindly increasing the congestion window.

Question 2

The major enhancement in **TcpSlok** is the replacement of traditional Slow Start with **TCP Hystart**.

- **Problem with NewReno Slow Start:**
  - Exponential growth continues until packet loss
  - Causes queue buildup and high RTT
  - Late congestion detection
- **Hystart Approach:** Hystart detects congestion early using **RTT measurements** instead of waiting for packet loss.
- **Implementation Logic:**
  - Track minimum RTT (`minRTT`)
  - Monitor current RTT (`lastRTT`)
  - If: $currentRTT > minRTT + threshold$ → Exit Slow Start early
- **Benefits:**
  - Prevents buffer overflow
  - Reduces packet loss

- Stabilizes RTT
- Smooth transition to congestion avoidance

```
~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✓ at 18:06:14
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=4 --mode=mixed"
Flow 0 Throughput: 0.477397 Mbps
Flow 1 Throughput: 0.377106 Mbps
Flow 2 Throughput: 0.480732 Mbps
Flow 3 Throughput: 0.473109 Mbps
Jain Fairness Index: 0.990879

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✓ took 5s at 18:13:20
```

```
~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✓ took 5s at 18:13:20
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=8 --mode=mixed"
Flow 0 Throughput: 0.211541 Mbps
Flow 1 Throughput: 0.205586 Mbps
Flow 2 Throughput: 0.21178 Mbps
Flow 3 Throughput: 0.216306 Mbps
Flow 4 Throughput: 0.252992 Mbps
Flow 5 Throughput: 0.248942 Mbps
Flow 6 Throughput: 0.21702 Mbps
Flow 7 Throughput: 0.243701 Mbps
Jain Fairness Index: 0.993747

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✓ took 4s at 18:14:07
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=16 --mode=mixed"
Flow 0 Throughput: 0.116967 Mbps
Flow 1 Throughput: 0.112441 Mbps
Flow 2 Throughput: 0.119588 Mbps
Flow 3 Throughput: 0.117444 Mbps
Flow 4 Throughput: 0.111726 Mbps
Flow 5 Throughput: 0.114108 Mbps
Flow 6 Throughput: 0.115061 Mbps
Flow 7 Throughput: 0.122208 Mbps
Flow 8 Throughput: 0.118873 Mbps
Flow 9 Throughput: 0.10458 Mbps
Flow 10 Throughput: 0.102912 Mbps
Flow 11 Throughput: 0.10982 Mbps
Flow 12 Throughput: 0.108629 Mbps
Flow 13 Throughput: 0.115776 Mbps
Flow 14 Throughput: 0.116252 Mbps
Flow 15 Throughput: 0.107438 Mbps
Jain Fairness Index: 0.997794

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✓ took 4s at 18:14:25
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=20 --mode=mixed"
Flow 0 Throughput: 0.0876658 Mbps
Flow 1 Throughput: 0.0971947 Mbps
Flow 2 Throughput: 0.0831396 Mbps
Flow 3 Throughput: 0.0871893 Mbps
Flow 4 Throughput: 0.0914773 Mbps
Flow 5 Throughput: 0.0936213 Mbps
Flow 6 Throughput: 0.0936213 Mbps
Flow 7 Throughput: 0.0936213 Mbps
Flow 8 Throughput: 0.0862364 Mbps
Flow 9 Throughput: 0.0933831 Mbps
Flow 10 Throughput: 0.0948124 Mbps
Flow 11 Throughput: 0.0940978 Mbps
Flow 12 Throughput: 0.0952889 Mbps
Flow 13 Throughput: 0.0926684 Mbps
Flow 14 Throughput: 0.0955271 Mbps
Flow 15 Throughput: 0.0898098 Mbps
Flow 16 Throughput: 0.0819484 Mbps
Flow 17 Throughput: 0.090048 Mbps
Flow 18 Throughput: 0.0807573 Mbps
Flow 19 Throughput: 0.0869511 Mbps
Jain Fairness Index: 0.997316

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✓ took 4s at 18:14:42
```

Question 3

- **Topology Used:** A **Dumbbell Topology** was used (from demo.cc):
  - Multiple senders (left nodes)
  - Multiple receivers (right nodes)
  - A bottleneck link (2 Mbps)
- **Modifications:**
  - All flows converted to TCP
  - Two configurations tested:
    1. All flows using **TcpNewReno**
    2. All flows using **TcpSlok**
- **Metrics Evaluated:**
  A. Throughput
  B. Congestion Window (cwnd)
  C. Slow Start Threshold (ssthresh)
  D. RTT variation

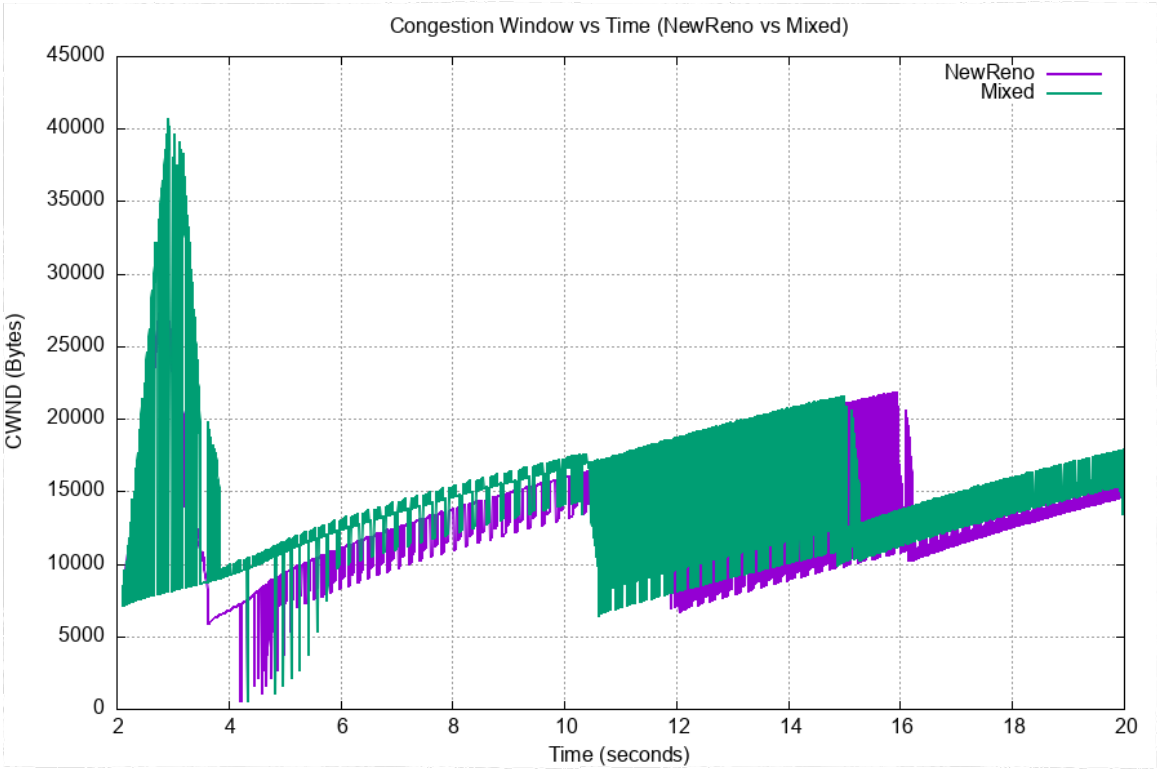
Question 4

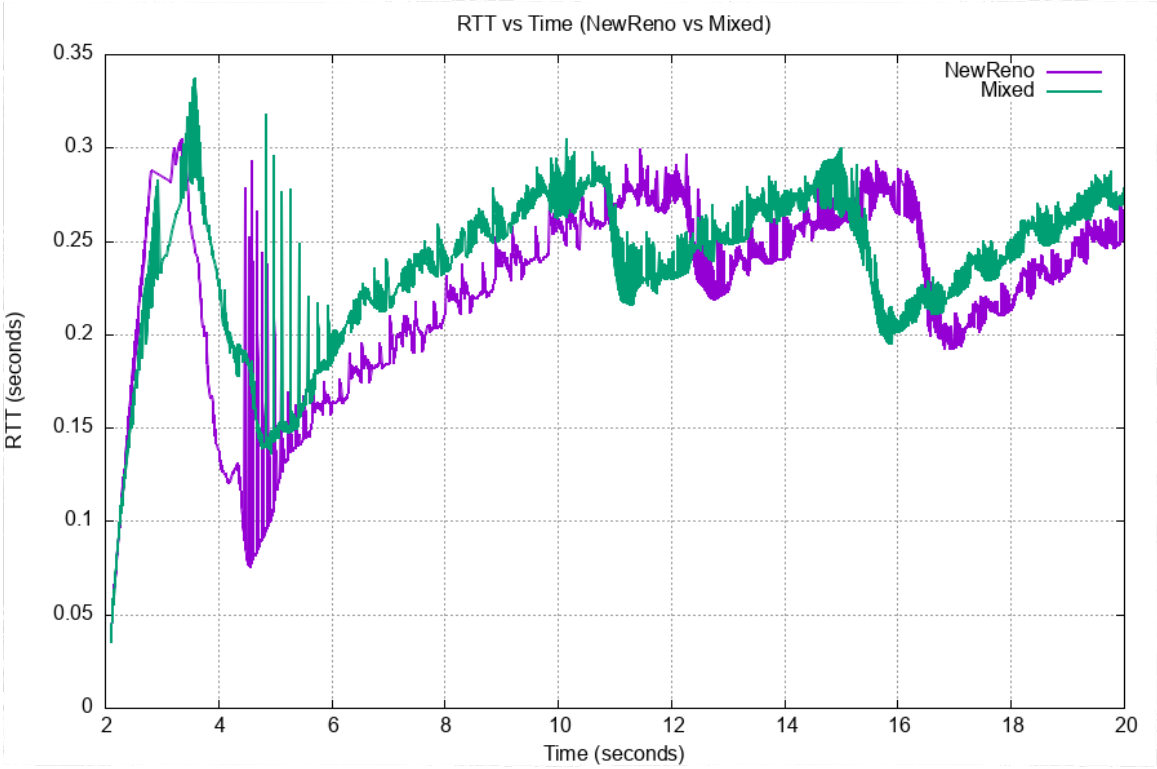
Number of flows	**4**	**8**	**16**	**20**
NewReno & TCPSlok	0.990879	0.993747	0.997794	0.997316
NewReno	0.979283	0.9941	0.998228	0.995987

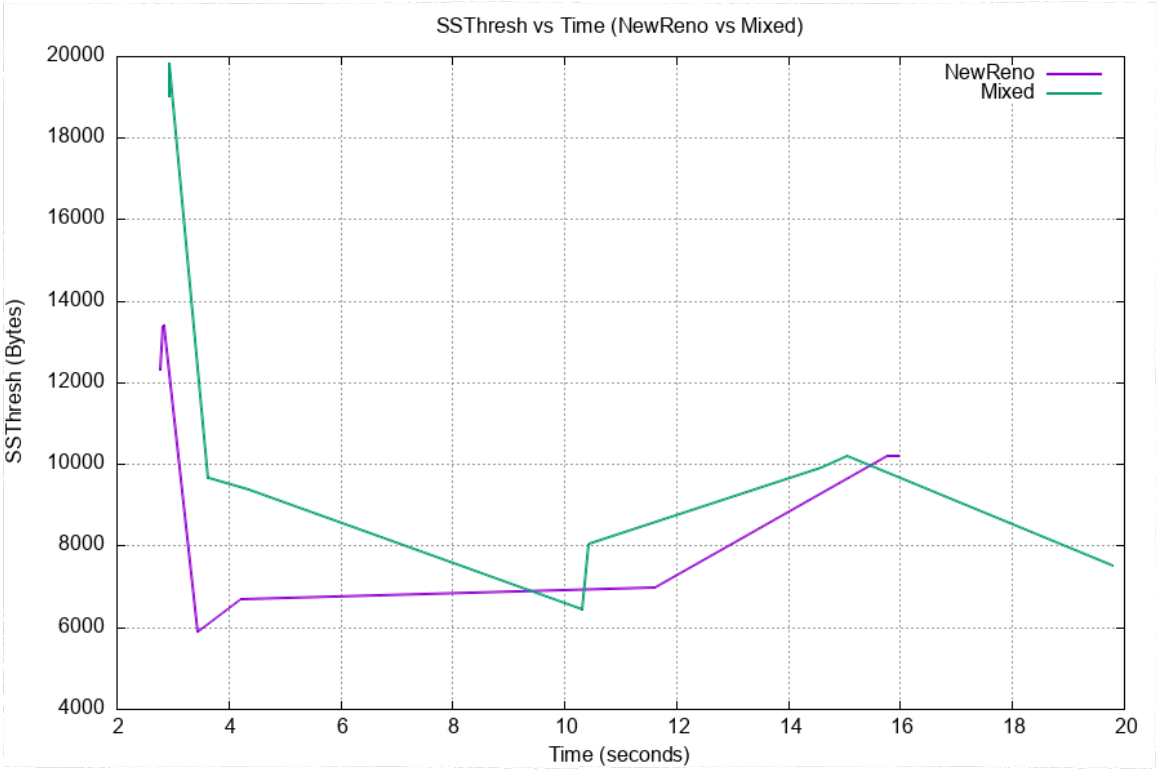
```
~/Downloads/ns-allinone-3.46.1/ns-3.46.1                                  ✔ at 18:16:56
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=4 --mode=newreno"
Flow 0 Throughput: 0.375915 Mbps
Flow 1 Throughput: 0.400452 Mbps
Flow 2 Throughput: 0.517657 Mbps
Flow 3 Throughput: 0.520277 Mbps
Jain Fairness Index: 0.979283

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                  ✔ took 4s at 18:18:18
```

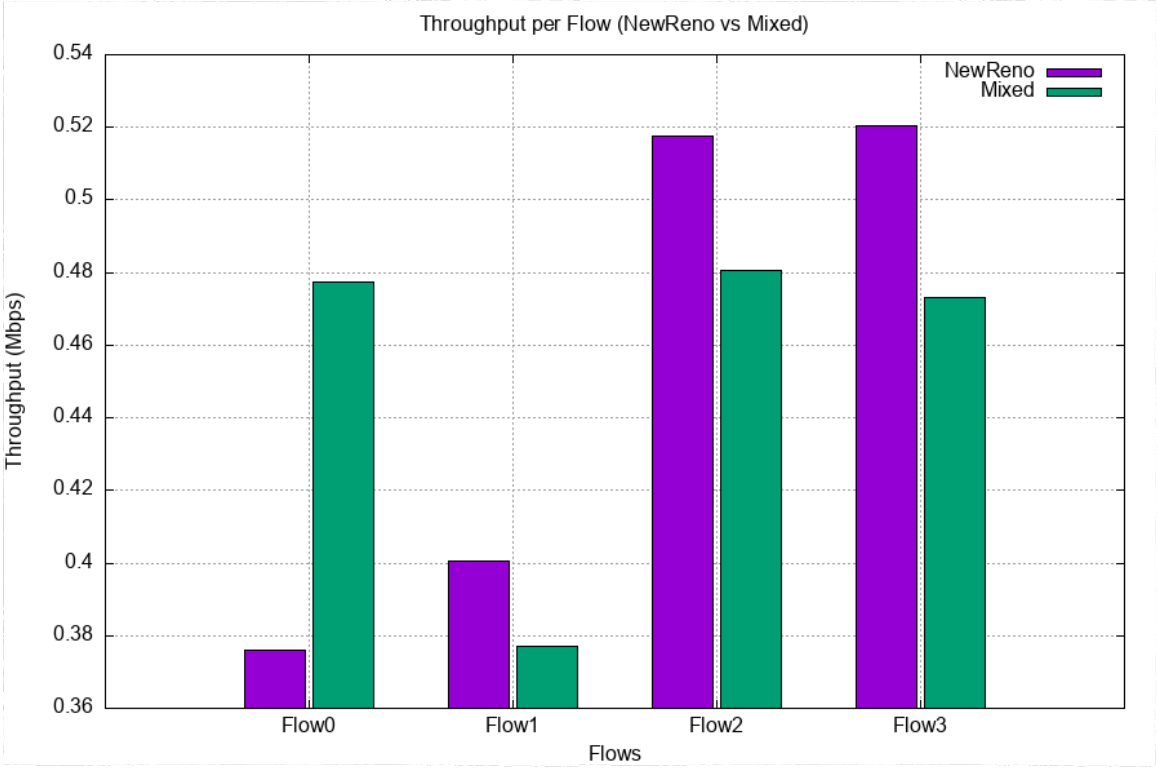
```
~/Downloads/ns-allinone-3.46.1/ns-3.46.1                  ✔ took 4s at 18:18:18
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=8 --mode=newreno"
Flow 0 Throughput: 0.243225 Mbps
Flow 1 Throughput: 0.221308 Mbps
Flow 2 Throughput: 0.210588 Mbps
Flow 3 Throughput: 0.233696 Mbps
Flow 4 Throughput: 0.229646 Mbps
Flow 5 Throughput: 0.251324 Mbps
Flow 6 Throughput: 0.192007 Mbps
Flow 7 Throughput: 0.226549 Mbps
Jain Fairness Index: 0.9941

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                  ✔ took 4s at 18:18:53
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=16 --mode=newreno"
Flow 0 Throughput: 0.11792 Mbps
Flow 1 Throughput: 0.116014 Mbps
Flow 2 Throughput: 0.115538 Mbps
Flow 3 Throughput: 0.116014 Mbps
Flow 4 Throughput: 0.116252 Mbps
Flow 5 Throughput: 0.117444 Mbps
Flow 6 Throughput: 0.116014 Mbps
Flow 7 Throughput: 0.112679 Mbps
Flow 8 Throughput: 0.117682 Mbps
Flow 9 Throughput: 0.106247 Mbps
Flow 10 Throughput: 0.11125 Mbps
Flow 11 Throughput: 0.114585 Mbps
Flow 12 Throughput: 0.109344 Mbps
Flow 13 Throughput: 0.114108 Mbps
Flow 14 Throughput: 0.107915 Mbps
Flow 15 Throughput: 0.100292 Mbps
Jain Fairness Index: 0.998228

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                  ✔ took 4s at 18:19:03
./ns3 run "scratch/tcp-slok-dumbbell --nFlows=20 --mode=newreno"
Flow 0 Throughput: 0.094336 Mbps
Flow 1 Throughput: 0.102197 Mbps
Flow 2 Throughput: 0.0907627 Mbps
Flow 3 Throughput: 0.100053 Mbps
Flow 4 Throughput: 0.0850453 Mbps
Flow 5 Throughput: 0.0907627 Mbps
Flow 6 Throughput: 0.09648 Mbps
Flow 7 Throughput: 0.0983858 Mbps
Flow 8 Throughput: 0.0867129 Mbps
Flow 9 Throughput: 0.0890951 Mbps
Flow 10 Throughput: 0.0824249 Mbps
Flow 11 Throughput: 0.0817102 Mbps
Flow 12 Throughput: 0.0895716 Mbps
Flow 13 Throughput: 0.0912391 Mbps
Flow 14 Throughput: 0.0914773 Mbps
Flow 15 Throughput: 0.0838542 Mbps
Flow 16 Throughput: 0.0931449 Mbps
Flow 17 Throughput: 0.0821867 Mbps
Flow 18 Throughput: 0.0933831 Mbps
Flow 19 Throughput: 0.0893333 Mbps
Jain Fairness Index: 0.995987

~/Downloads/ns-allinone-3.46.1/ns-3.46.1                  ✔ took 4s at 18:19:13
```

Question 5


Congestion Window vs Time (NewReno vs Mixed)
NewReno
Mixed
CWND (Bytes)
Time (seconds)
0
5000
10000
15000
20000
25000
30000
35000
40000
45000
2
4
6
8
10
12
14
16
18
20


RTT vs Time (NewReno vs Mixed)
NewReno
Mixed
RTT (seconds)
Time (seconds)
0
0.05
0.1
0.15
0.2
0.25
0.3
0.35
2
4
6
8
10
12
14
16
18
20


SSThresh vs Time (NewReno vs Mixed)
NewReno
Mixed
SSThresh (Bytes)
20000
18000
16000
14000
12000
10000
8000
6000
4000
2
4
6
8
10
12
14
16
18
20
Time (seconds)


Throughput per Flow (NewReno vs Mixed)
NewReno
Mixed
Throughput (Mbps)
0.54
0.52
0.5
0.48
0.46
0.44
0.42
0.4
0.38
0.36
Flow0
Flow1
Flow2
Flow3
Flows

Question 6

- Why **TcpSlok** Performs Better:
  1. Early Congestion Detection
     - Uses RTT instead of packet loss
     - Avoids entering congested state
  2. Reduced Packet Loss
     - Prevents buffer overflow in bottleneck link
  3. Stable Network Behavior
     - Smooth cwnd growth
     - Reduced oscillations
  4. Improved Fairness
     - Ensures equitable bandwidth sharing
     - Jain index closer to ideal (1)
- Trade-offs:
  - Slightly conservative compared to NewReno
  - May reduce peak cwnd in some cases
- Key Insight:

  "Replacing loss-based detection with delay-based detection leads to more efficient and stable TCP performance."